

Basics

Bit MANIPULATION

Module 0: Binary Number System Foundations

Before touching operators, you must be able to think in binary.

Topics:

- Decimal ↔ Binary conversion
- Binary place values
- Powers of 2
- Binary representation of numbers
- Signed vs Unsigned numbers
- 1's Complement
- 2's Complement
- Negative numbers in computers

Goals:

- Convert numbers mentally
- Recognize powers of 2 instantly
- Understand why $-x$ works in bit tricks

Problems:

- Power of Two
 - Number Complement
-

Module 1: Bitwise Operators

The foundation of everything.

Topics:

- AND (&)
- OR (|)
- XOR (^)

- NOT (~)
- Left Shift (<<)
- Right Shift (>>)

Goals:

- Predict output without compiler
- Understand bit-level effects

Must Know:

$a \& b$

$a | b$

$a \wedge b$

$\sim a$

$a \ll k$

$a \gg k$

Problems:

- Bitwise AND of Numbers Range
- XOR Operation in an Array

Module 2: Fundamental Bit Tricks

The most important module.

Topics:

Check kth bit

$(a \& (1 \ll b)) \neq 0$

Set kth bit

$a | (1 \ll b)$

Clear kth bit

$a \& \sim(1 \ll b)$

Toggle kth bit

$a \wedge (1 \ll b)$

Goals:

- Do these operations instantly

Problems:

- Single Number
 - Missing Number
-

Module 3: Important Patterns

Interview favorites.

Check odd/even

$(n \& 1)$

Power of 2

$(n \& (n-1)) == 0$

Remove rightmost set bit

$n \& (n-1)$

Extract rightmost set bit

$n \& (-n)$

Count set bits (Brian Kernighan)

```
while(n!=0){  
    n &= (n-1);  
    count++;  
}
```

Problems:

- Number of 1 Bits
 - Counting Bits
 - Power of Two
-

Module 4: XOR Mastery

This module alone solves many interview questions.

Properties:

$$a \oplus a = 0$$

$$a \oplus 0 = a$$

Topics:

- Single number
- Missing number
- Find duplicate
- Swap without temp
- Prefix XOR

Problems:

- Single Number
 - Missing Number
 - Find XOR of range
-

Module 5: Bitmasking Basics

Start representing sets using bits.

Example:

0001 -> {0}

0011 -> {0,1}

1111 -> {0,1,2,3}

Topics:

- Set representation
- Add element
- Remove element
- Membership checking

Problems:

- Subsets
 - Power Set
-

Module 6: Subset Generation

Extremely important for OA and CP.

Generate all subsets:

```
for(mask = 0; mask < (1<<n); mask++)
```

Topics:

- Enumerating subsets
- Enumerating submasks
- Power set generation

Problems:

- Subsets
 - Subsets II
-

Module 7: Advanced XOR Problems

Topics:

- Two unique numbers
- XOR basis intuition
- Maximum XOR

Problems:

- Single Number II
 - Single Number III
 - Maximum XOR of Two Numbers
-

Module 8: Bitmask Dynamic Programming

This is where Codeforces starts becoming interesting.

Topics:

- State compression
- DP with masks
- Traveling Salesman Problem

Problems:

- TSP
 - Partition problems
 - Assignment problems
-

Module 9: Advanced Bit Tricks

Topics:

- Gray Code
- Lowbit
- Fast modulo powers of 2
- Binary lifting intuition
- XOR basis

Problems:

- Gray Code
 - Beautiful Arrangement
-

Module 10: Competitive Programming Bitmasking

Topics:

- SOS DP

- Meet in the Middle
- Bitset optimization
- Inclusion Exclusion

Problems:

- Codeforces 1400-2200 bitmask problems
-

Module 11: Interview Bit Manipulation Patterns

Patterns used by:

- Google
- Amazon
- Microsoft
- Meta

Recognition guide:

- "Every element appears twice except one" → XOR
 - "Power of two" → $n \& (n-1)$
 - "Generate all combinations" → Bitmask
 - "Subset constraints" → Bit DP
-

Module 12: Expert Level Topics

Topics:

- XOR Basis
- Linear Algebra over Bits
- Gaussian Elimination on Bits
- Trie + XOR
- Bitset Optimization

0. Binary Basics

1. Operators
2. Fundamental Bit Tricks
3. Important Patterns
4. XOR Mastery
5. Bitmask Basics
6. Subset Generation
7. Advanced XOR
8. Bitmask DP
9. Advanced Tricks
10. CP Bitmasking
11. Interview Patterns
12. Expert Topics

Binary Decimal

1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15
10000	16

Notice:

1
10
100
1000
10000

These are powers of 2.

$$\begin{aligned}2^0 &= 1 \\2^1 &= 2 \\2^2 &= 4 \\2^3 &= 8 \\2^4 &= 16 \\2^5 &= 32\end{aligned}$$

Lesson 4: Powers of Two Recognition

You should instantly know:

$$\begin{aligned}2^0 &= 1 \\2^1 &= 2 \\2^2 &= 4 \\2^3 &= 8 \\2^4 &= 16 \\2^5 &= 32 \\2^6 &= 64 \\2^7 &= 128 \\2^8 &= 256 \\2^9 &= 512 \\2^{10} &= 1024\end{aligned}$$

Especially:

$$1024 \approx 10^3$$

$$\begin{aligned}2^{10} &= 1024 \\2^{20} &\approx 10^6 \\2^{30} &\approx 10^9\end{aligned}$$

Signed Numbers, 1's Complement, and 2's Complement

1's Complement

To get negative number:

Flip every bit

Example:

$5 = 0101$

Flip bits:

1010

Therefore:

$-5 = 1010$

$+0 = 0000$

$-0 = 1111$

Still two zeros.

Bad design.

2's Complement (Used Everywhere)

Modern computers use this.

To get negative number:

Step 1

Flip all bits.

Step 2

Add 1.

Instead of:

Flip

Add 1

Remember:

$-n = (\sim n) + 1$

$-n == (\sim n) + 1$

Famous Trick

$n \& (-n)$

returns:

Rightmost set bit

$$\sim n = -(n + 1)$$

$$\sim 7 = -8$$

$$\sim 10 = -11$$

$$\sim 0 = -1$$

Things You Must Remember

Formula 1

$$-n = (\sim n) + 1$$

Formula 2

$$\sim n = -(n + 1)$$

Formula 3

$$n \& (-n)$$

Position Indexing

We always count from the right.

$$1 \ll k == 2^k$$

Powers of Two Revisited

Look:

$$1 \ll 0 = 1$$

$$1 \ll 1 = 2$$

$$1 \ll 2 = 4$$

$$1 \ll 3 = 8$$

$$1 \ll 4 = 16$$

$$1 \ll 5 = 32$$

Four Facts Every Expert Knows

Fact 1

$$a \ll b = a * 2^b$$

Fact 2

$$a \gg b = \text{floor}(a / 2^b)$$

Fact 3

$$1 \ll b$$

creates a number with only the bth bit set.

Fact 4

$$(a \gg b) \& 1$$

Trick 1: Check kth Bit

Question:

Is bit k set?

Formula

$$(n \& (1 \ll k)) \neq 0$$

Trick 2: Set kth Bit

Question:

Make kth bit = 1

Formula

$n | (1 \ll k)$

Trick 3: Clear kth Bit

Question:

Make kth bit = 0

Formula

$n \& \sim(1 \ll k)$

Trick 4: Toggle kth Bit

Question:

If 1 → make 0

If 0 → make 1

Formula

$n \wedge (1 \ll k)$

Pattern 1: Odd or Even

Trick

$(n \& 1)$

Why?

The last bit determines odd/even.

8 = 1000 -> even

10 = 1010 -> even

7 = 0111 -> odd

13 = 1101 -> odd

Notice:

Even => last bit = 0

Odd => last bit = 1

So:

if $((n \& 1) == 1)$

means odd.

Examples

$7 \& 1 = 1$

Odd.

$10 \& 1 = 0$

Even.

Pattern 2: Power of Two

One of the most famous interview questions.

Examples:

1 = 0001

2 = 0010

4 = 0100

8 = 1000

16 = 10000

Observation:

Every power of 2 has exactly **one set bit**.

Trick

$(n \& (n - 1)) == 0$

and

$n > 0$

must also hold.

Final:

$n > 0 \&\& (n \& (n - 1)) == 0$

Why?

Example:

8 = 1000

7 = 0111

AND:

1000

0111

0000

Result:

$8 \& 7 = 0$

Important Observation

$n \& (n - 1)$

removes the rightmost set bit.

This observation leads to several tricks.

Pattern 3: Remove Rightmost Set Bit

Formula

$n \& (n - 1)$

Pattern 4: Extract Rightmost Set Bit

Formula

$n \& (-n)$

Pattern 5: Count Set Bits (Naive)

Question:

How many 1s are present?

Example:

13 = 1101

Answer:

3

Method 1

```
int count = 0;
```

```
while (n > 0) {
```

```
    count += (n & 1);
```

```
n >>= 1;  
}
```

Complexity:

O(number of bits)

For int:

O(32)

Pattern 6: Brian Kernighan Algorithm

One of the most important interview tricks.

Core Idea

Every operation:

```
n = n & (n - 1);
```

removes one set bit.

So:

```
int count = 0;
```

```
while (n != 0) {  
    n = n & (n - 1);  
    count++;  
}
```

Example

13 = 1101

Step 1:

1101

1100

1100

Count = 1

Step 2:

1100

1011

1000

Count = 2

Step 3:

1000

0111

0000

Count = 3

Stop.

Answer:

3

Complexity

Instead of:

$O(32)$

it becomes:

$O(\text{number of set bits})$

which is often faster.

Question	Trick
Odd/Even	$n \& 1$
Power of 2	$n > 0 \&\& (n \& (n-1)) == 0$
Remove rightmost set bit	$n \& (n-1)$
Extract rightmost set bit	$n \& (-n)$
Count set bits	Brian Kernighan

How to Build the Mask?

Method 1:

Keep shifting.

```
int mask = 1;
```

```
while(mask < num){  
    mask = (mask << 1) | 1;  
}
```

XOR

Bitwise XOR Manipulation Table

Logic Overview

The Bitwise XOR (Exclusive OR) operator ($\wedge$) compares each bit of its first operand to the corresponding bit of its second operand. If the bits are different (one is 0 and the other is 1), the resulting bit is 1. If the bits are the same (both 0 or both 1), the resulting bit is 0.

Truth Table for XOR

The following table defines the fundamental logic of the XOR operation:

Input A	Input B	Output (A ^ B)
0	0	0
0	1	1
1	0	1
1	1	0

SAME $\rightarrow$ 0

DIFFERENT $\rightarrow$ 1

Lesson 2: The 4 Golden Properties

These 4 properties solve most XOR questions.

Property 1

$$a \oplus a = 0$$

Example:

$$7 \oplus 7 = 0$$

Property 2

$$a \oplus 0 = a$$

Example:

$$13 \oplus 0 = 13$$

Property 3

$$a \oplus b \oplus a = b$$

Because:

$$(a \oplus a) \oplus b$$

$$0 \oplus b$$

$$b$$

Property 4

Order doesn't matter.

$$a \oplus b == b \oplus a$$

and

$$(a \oplus b) \oplus c$$

=

$a \wedge (b \wedge c)$

Module 5: Bitmasking Basics

What is a Bitmask?

A bitmask is simply an integer where each bit represents some state

Why Bitmasking?

Suppose you have:

$\{A, B, C, D\}$

Normally you might use:

`HashSet<Character>`

With bitmasking:

`0000` → $\{\}$

`0001` → $\{A\}$

`0010` → $\{B\}$

`0011` → $\{A,B\}$

`0100` → $\{C\}$

...

`1111` → $\{A,B,C,D\}$

A single integer stores the entire set.

The Golden Mapping

For a set of size n :

bit $i \leftrightarrow$ element i

Example:

$0 \rightarrow A$

$1 \rightarrow B$

$2 \rightarrow C$

$3 \rightarrow D$

Then:

1101

means:

A present

B absent

C present

D present

Operation 1: Add an Element

Add element k .

Example:

Add C.

$\text{mask} \mid= (1 \ll k);$

This is exactly the "set kth bit" operation you already know.

Example:

mask = 1001

Add bit 1:

1001

0010

1011

Operation 2: Remove an Element

Remove element k .

mask $\&= \sim(1 \ll k);$

Example:

1011

Remove bit 1:

1011

1101

1001

Operation 3: Check Membership

Is element k present?

$(\text{mask} \& (1 \ll k)) \neq 0$

Example:

mask = 1011

Check bit 3:

1011

1000

1000

Present.

Operation 4: Toggle Membership

$\text{mask} \wedge= (1 \ll k);$

Present $\rightarrow$ absent

Absent $\rightarrow$ present

Most Important Insight

A mask is just a subset.

Suppose:

A B C

Three elements.

Possible subsets:

000 $\rightarrow \{\}$

001 $\rightarrow \{A\}$

010 $\rightarrow \{B\}$

011 $\rightarrow \{A,B\}$

100 $\rightarrow \{C\}$

101 $\rightarrow \{A,C\}$

110 $\rightarrow \{B,C\}$

111 $\rightarrow \{A,B,C\}$

Notice:

$2^3 = 8$ subsets

For:

n elements

Total subsets:

2^n

This fact drives almost all bitmasking problems.

Generating All Subsets

Suppose:

nums = [1,2,3]

There are:

$2^3 = 8$

subsets.

We generate masks:

```
for(int mask = 0; mask < (1 << n); mask++)
```

For:

n = 3

mask = 0 → 000

mask = 1 → 001

mask = 2 → 010

mask = 3 → 011

mask = 4 → 100

mask = 5 → 101

mask = 6 → 110

mask = 7 → 111

Each mask corresponds to one subset.

Example

Array:

[1,2,3]

Mask:

101

Interpret bits:

bit 0 = 1 → take 1

bit 1 = 0 → skip 2

bit 2 = 1 → take 3

Subset:

[1,3]

Interview Recognition

When you see:

- Generate all subsets
- Power set
- Small n (≤ 20)
- Pick/Don't Pick choices

Think:

```
for(mask = 0; mask < (1<<n); mask++)
```

This is one of the most important patterns in DSA.

Problems to Practice After This Module

1. Subsets
 2. Subsets II
 3. Letter Case Permutation
 4. Maximum Product of Word Lengths
-

Before Module 6

Make sure you can answer:

For:

```
nums = [1,2,3]
```

What subset does each mask represent?

000

001

010

011

100

101

110

111

If you can convert masks $\leftrightarrow$ subsets comfortably, you're ready for **Module 6: Subset Enumeration**, where we'll master the most important bitmasking loop used in LeetCode and Codeforces.

Subset Enumeration

Core Idea

For n elements:

Each element:

Take

or

Don't Take

Therefore:

2^n

possible subsets.

We represent each subset using a mask.

Example

nums = [10,20,30]

$n = 3$

Masks:

000

001

010

011

100

101

110

111

8 masks = 8 subsets.

```
for(int mask=0; mask<(1<<n); mask++){  
    for(int i=0; i<n; i++){  
        if((mask & (1<<i)) != 0){  
            // nums[i] belongs to this subset  
        }  
    }  
}
```

Pattern 4: Enumerating Only Chosen Elements

Instead of checking all n bits, advanced CP solutions often do:

```
while(mask != 0){  
    int bit = mask & (-mask);  
    ...  
    mask -= bit;  
}  
class Solution {  
    public List<List<Integer>> subsets(int[] nums) {  
  
        List<List<Integer>> result = new ArrayList<>();
```

```
int n = nums.length;
```

```
for (int mask = 0; mask < (1 << n); mask++) {
```

```
    List<Integer> subset = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) {
```

```
        if ((mask & (1 << i)) != 0) {
```

```
            subset.add(nums[i]);
```

```
        }
```

```
    }
```

```
    result.add(subset);
```

```
}
```

```
return result;
```

```
}
```

```
}
```

Bitmask DP

Module 8: Bitmask DP (State Compression DP)

First: What Problem Does Bitmask DP Solve?

Suppose:

$n = 15$

and every item can be:

taken

not taken

Total states:

2^{15}

$\approx 32,768$

That's small enough to store.

Instead of:

$dp[i][j][k][\dots]$

we compress the state into:

$Dp[mask]$

Classic Example: Traveling Salesman Problem (TSP)

Cities:

A B C D

Goal:

Visit every city exactly once with minimum cost.

Normally:

Which cities have been visited?

Current city?

Two pieces of information.

Represent visited cities using a bitmask.

Example:

1011

means:

A visited

B visited

C not visited

D visited

DP State:

$dp[mask][city]$

Meaning:

Minimum cost to visit cities in mask and end at city.

This is the most famous state-compression DP.

How to Recognize Bitmask DP?

Whenever you see:

Signal 1

$n \leq 20$

Huge clue.

Signal 2

Visited set

Chosen set

Assigned set

Huge clue.

Signal 3

Need minimum/maximum answer

over subsets.

Think DP.

Signal 4

Visit all nodes

Think TSP-style DP.

Pattern 1: DP Over Subsets

State:

`dp[mask]`

Meaning:

Answer for the subset represented by `mask`.

Generic Template

```
for (int mask = 0; mask < (1 << n); mask++) {  
  
    for (int i = 0; i < n; i++) {  
  
        if ((mask & (1 << i)) == 0) {  
  
            int newMask = mask | (1 << i);  
  
            dp[newMask] = ...  
        }  
    }  
}
```

Recognition

Think this pattern when:

Choose some elements

Assign jobs

Build subsets

Partition elements

Examples:

- Partition to K Equal Sum Subsets
 - Assignment problems
 - Minimum subset cost
-

Pattern 2: State Compression + Current Position

State:

`dp[mask][i]`

Meaning:

We have visited nodes in `mask` and currently stand at node `i`.

Generic Template

```
for (int mask = 0; mask < (1 << n); mask++) {  
  
    for (int u = 0; u < n; u++) {  
  
        if ((mask & (1 << u)) == 0)  
            continue;  
  
        for (int v = 0; v < n; v++) {  
  
            if ((mask & (1 << v)) != 0)  
                continue;  
  
            int newMask = mask | (1 << v);  
  
            dp[newMask][v] =  
                Math.min(  
                    dp[newMask][v],  
                    dp[mask][u] + cost[u][v]  
                );  
        }  
    }  
}
```

Recognition

Think this pattern when:

Visit all cities

Visit all nodes

Shortest Hamiltonian path

Traveling Salesman

Examples:

- TSP
- Hamiltonian Path
- Visiting all houses problems

Pattern 3: Memoization + Bitmask

Most common in interviews.

State:

dfs(mask)

or

dfs(mask, state)

Memoized.

Generic Template

```
Map<Integer, Integer> memo;
```

```
int dfs(int mask) {
```

```
    if (base_case)
        return answer;
```

```
    if (memo.containsKey(mask))
        return memo.get(mask);
```

```
    int ans = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if ((mask & (1 << i)) == 0) {
```

```
            ans = Math.max(
                ans,
                dfs(mask | (1 << i))
            );
```

```
        }
    }
```

```
    memo.put(mask, ans);
```

```
return ans;  
}
```

Recognition

Think:

Game states

Can I Win

Pick numbers

Choose moves

Examples:

- Can I Win
 - Many minimax + bitmask questions
-

The Universal Mental Model

Every Bitmask DP problem asks:

Question 1

What does the mask represent?

Examples:

Visited cities

Chosen elements

Assigned jobs

Used numbers

Question 2

What additional information is needed?

Examples:

Current city

Current sum

Current player
Current score

Question 3

How do I move to the next state?

Usually:

$\text{newMask} = \text{mask} | (1 \ll i)$

Choose element i .

My 4-Step Framework

When you see a Bitmask DP problem:

Step 1

Ask:

Can a subset be represented by bits?

If yes:

mask

Step 2

Define state.

Examples:

$\text{dp}[\text{mask}]$
 $\text{dp}[\text{mask}][i]$
 $\text{dfs}(\text{mask})$
 $\text{dfs}(\text{mask}, \text{sum})$

Step 3

Find transition.

Usually:

$\text{mask} | (1 \ll i)$

or

$\text{mask} \wedge (1 \ll i)$

Step 4

Find base case.

Usually:

$\text{mask} == (1 \ll n) - 1$

Meaning:

Everything chosen.

Example: Tiny Problem

Choose all numbers exactly once.

$\text{nums} = [1, 2, 3]$

Goal:

Maximum sum.

State:

$\text{dfs}(\text{mask})$

Transition:

Choose unused number.

$\text{newMask} = \text{mask} | (1 \ll i)$

Base:

mask == 111

Done.

This exact thinking scales to very hard problems.

The Most Important Recognition Signal

When you see:

$n \leq 15$

$n \leq 18$

$n \leq 20$

CheatSheet

Bit Manipulation Interview Cheat Sheet (Easy → Medium)

This is the condensed version of everything you need for most interview and OA bit-manipulation problems.

1. Binary Basics

Powers of 2

$$2^0 = 1$$

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^9 = 512$$

$$2^{10} = 1024$$

Binary Place Values

Bit Position

3 2 1 0

8 4 2 1

Example:

1101

$$= 8 + 4 + 1$$

= 13

2. Operators

AND (&)

$1 \& 1 = 1$

$1 \& 0 = 0$

$0 \& 1 = 0$

$0 \& 0 = 0$

Meaning:

Both bits must be 1.

OR (|)

$1 | 1 = 1$

$1 | 0 = 1$

$0 | 1 = 1$

$0 | 0 = 0$

Meaning:

At least one bit is 1.

XOR (^)

$1 \wedge 1 = 0$

$1 \wedge 0 = 1$

$0 \wedge 1 = 1$

$0 \wedge 0 = 0$

Meaning:

Bits are different.

NOT (~)

$$\sim n = -(n + 1)$$

Examples:

$$\sim 5 = -6$$

$$\sim 10 = -11$$

$$\sim 0 = -1$$

Left Shift

$$n \ll k$$

Formula:

$$n * (2^k)$$

Examples:

$$5 \ll 1 = 10$$

$$5 \ll 2 = 20$$

Right Shift

$$n \gg k$$

Formula:

$$\text{floor}(n / 2^k)$$

Examples:

$$20 \gg 1 = 10$$

$$20 \gg 2 = 5$$

3. Most Important Identity

$$1 \ll k$$

Creates a number having only kth bit set.

Examples:

$1 \ll 0 = 0001$

$1 \ll 1 = 0010$

$1 \ll 2 = 0100$

$1 \ll 3 = 1000$

4. Check / Set / Clear / Toggle

Check kth Bit

$(n \& (1 \ll k)) \neq 0$

Alternative:

$((n \gg k) \& 1) == 1$

Set kth Bit

$n \mid (1 \ll k)$

Clear kth Bit

$n \& \sim(1 \ll k)$

Toggle kth Bit

$n \wedge (1 \ll k)$

5. Odd / Even

Check Odd

$(n \& 1) == 1$

Check Even

`(n & 1) == 0`

6. Power of Two

Observation

A power of 2 has exactly one set bit.

1 = 0001

2 = 0010

4 = 0100

8 = 1000

16 = 10000

Check Power of Two

`n > 0 && (n & (n - 1)) == 0`

7. Famous Bit Tricks

Remove Rightmost Set Bit

`n & (n - 1)`

Example:

1100

↓

1000

Extract Rightmost Set Bit

`n & (-n)`

Examples:

$$12 \& (-12) = 4$$

$$10 \& (-10) = 2$$

$$40 \& (-40) = 8$$

Two's Complement

$$-n = (\sim n) + 1$$

8. Count Set Bits

Brian Kernighan

```
int count = 0;
```

```
while(n != 0){  
    n &= (n - 1);  
    count++;  
}
```

Complexity:

$O(\text{number of set bits})$

Built-in

`Integer.bitCount(n)`

9. XOR Mastery

Golden Properties

$$a \oplus a = 0$$

$$a \oplus 0 = a$$

$$a \oplus b \oplus a = b$$

XOR is Commutative

$$a \oplus b = b \oplus a$$

XOR is Associative

$$(a \oplus b) \oplus c$$

=

$$a \oplus (b \oplus c)$$

10. XOR Problem Recognition

Every Element Appears Twice Except One

Use:

```
ans ^= nums[i];
```

Problem:

Single Number

Missing Number

XOR indices and values.

Problem:

Missing Number

Hamming Distance

$x \wedge y$

Then count set bits.

Problem:

Hamming Distance

Two Unique Numbers

Use:

$xor = \text{all numbers XOR}$

$diffBit = xor \& (-xor)$

Problem:

Single Number III

One Number Appears Once, Others Thrice

Count bits.

Use:

$count \% 3$

Problem:

Single Number II

11. Number Complement Pattern

Problem:

Number Complement

Formula:

$(\sim \text{num}) \& \text{mask}$

Mask:

```
int mask = 1;
```

```
while(mask < num)
    mask = (mask << 1) | 1;
```

12. Bitmasking Basics

Represent Set

0000 -> {}

0001 -> {0}

0011 -> {0,1}

1111 -> {0,1,2,3}

Add Element

```
mask |= (1 << k);
```

Remove Element

```
mask &= ~(1 << k);
```

Check Element

```
(mask & (1 << k)) != 0
```

Toggle Element

```
mask ^= (1 << k);
```

13. Generate All Subsets

Core Loop

```
for(int mask = 0; mask < (1 << n); mask++)
```

Check Included Elements

```
for(int i = 0; i < n; i++){  
  
    if((mask & (1 << i)) != 0){  
  
        // nums[i] belongs to subset  
    }  
}
```

Complexity

$O(n \cdot 2^n)$ $O(n \cdot 2^n)$ $O(n \cdot 2^n)$

14. Pattern Recognition Table

Problem Clue	Pattern
Odd / Even	$n \& 1$
Power of 2	$n \& (n-1)$
Count set bits	Brian Kernighan
Unique element	XOR
Missing number	XOR
Hamming distance	XOR + bit count

Two unique numbers XOR + x & $(-x)$

Number complement $(\sim n)$ & mask

Generate subsets Bitmask

$n \leq 20$ Consider bitmasking

FormulaSheet

Bit Manipulation Ultimate Formula Sheet (Easy → Medium)

This comprehensive guide serves as a quick reference for technical interviews and competitive programming, covering fundamental bitwise operations, common patterns, and advanced tricks.

Core Bitwise Operations

Topic	Formula / Trick	Purpose
Check Odd	$(n \ \& \ 1) == 1$	Number is odd
Check Even	$(n \ \& \ 1) == 0$	Number is even
kth Bit Mask	$1 \ll k$	Create mask with only kth bit set
Check kth Bit	$(n \ \& \ (1 \ll k)) \neq 0$	Is kth bit set?
Set kth Bit	$n \mid (1 \ll k)$	Set kth bit
Clear kth Bit	$n \ \& \ \sim(1 \ll k)$	Make kth bit = 0
Toggle kth Bit	$n \ \wedge \ (1 \ll k)$	Flip kth bit
Multiply by 2^k	$n \ll k$	Fast multiplication
Divide by 2^k	$n \gg k$	Fast division

Power of 2 Patterns

Topic	Formula	Purpose
Power of 2 Check	<code>n > 0 && (n & (n - 1)) == 0</code>	Check if <code>n</code> is a power of 2
Remove Rightmost Set Bit	<code>n & (n - 1)</code>	Deletes lowest set bit
Extract Rightmost Set Bit	<code>n & (-n)</code>	Gets lowest set bit
Negative Number	<code>-n = (~n) + 1</code>	Two's complement representation
NOT Operator	<code>~n = -(n + 1)</code>	Quick calculation of bitwise NOT

Set Bit Counting

Topic	Formula	Purpose
Built-in Count	<code>Integer.bitCount(n)</code>	Java built-in function
Brian Kernighan's	<code>while(n!=0) { n &= (n-1); cnt++; }</code>	Efficient set bit counting
LeetCode 191	Number of Set Bits	Count 1s in binary
LeetCode 338	Counting Bits	DP + bit tricks for range [0, n]

XOR Mastery

Property	Formula	Description
Same Cancels	$a \oplus a = 0$	XORing a number with itself results in 0
Identity	$a \oplus 0 = a$	XORing with 0 returns the original value
Rearrangement	$a \oplus b \oplus a = b$	Commutative and associative
Commutative	$a \oplus b = b \oplus a$	Order does not matter
Associative	$(a \oplus b) \oplus c = a \oplus (b \oplus c)$	Grouping does not matter

XOR Problem Recognition

Problem Type	Trick
One unique, others twice	XOR all elements
Missing Number	XOR indices + values
Hamming Distance	<code>bitCount(x ^ y)</code>
Two Unique Numbers	<code>xor & (-xor)</code> to partition into two groups
One Unique, others thrice	Bit count modulo 3

Bitmasking & Subsets

Operation / Topic	Formula
Add element	<code>`mask</code>
Remove element	<code>mask &= ~(1 << k)</code>
Check element	<code>(mask & (1 << k)) != 0</code>
Toggle element	<code>mask ^= (1 << k)</code>
Generate All Subsets	<code>for(mask=0; mask < (1 << n); mask++)</code>
Check inclusion	<code>(mask & (1 << i)) != 0</code>
Number of Subsets	<code>2^n</code>
Subset Size	<code>Integer.bitCount(mask)</code>

Interview Pattern Recognition

If You See...	Think...
Odd / Even	<code>n & 1</code> ▾
Power of 2	<code>n & (n - 1)</code> ▾
Count bits	Brian Kernighan's Algorithm ▾
Unique element	XOR ▾
Missing number	XOR ▾

If You See...	Think...
Hamming distance	XOR + bitCount ▾
Two unique numbers	XOR + n & (-n) ▾
Generate all subsets	Bitmasking ▾
$n \leq 20$	Bitmasking candidate ▾
Pick / Don't Pick	Subsets / Bitmask ▾

Top 10 Must-Memorize Formulas

#	Formula
1	$n \& 1$
2	$n \& (1 \ll k)$
3	$\sim n$
4	$n \& \sim(1 \ll k)$
5	$n \wedge (1 \ll k)$
6	$n \& (n - 1)$
7	$n \& (-n)$
8	$a \wedge a = 0$
9	$a \wedge 0 = a$

#	Formula
10	<pre>for(mask=0; mask < (1 << n); mask++)</pre>